

Reviewer Pack — Persistent H1 Lifespan Bounds Resolution Width on Tseitin Fo...

Entry c6142ec20dd6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 18:11:25 UTC

Persistent H1 Lifespan Bounds Resolution Width on Tseitin Formulas

Entry ID: c6142ec20dd6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 18:11:25 UTC

1. Conjecture

Field A (mathematical branch): Persistent Homology (Topological Data Analysis on filtered simplicial complexes) **Field B** (complexity object): Resolution proof width of Tseitin formulas

Statement:

For a 3-regular graph G with odd charge, build the Vietoris-Rips filtration on the clause-incidence point cloud $P(\varphi_G) \subset \mathbb{R}^n$ (each clause is a ± 1 vector on its variables), and let $L1(\varphi_G)$ be the sum of lifespans (death – birth) of all H1 bars truncated at scale $2\sqrt{3}$. Then there exists an absolute constant $C \geq 1$ such that the resolution proof width $w(\varphi_G)$ satisfies $w(\varphi_G) \leq C \cdot (L1(\varphi_G) + \log n)$, and equivalently $L1(\varphi_G) = \Omega(w(\varphi_G)) - O(\log n)$. A single 3-regular Tseitin instance with $L1(\varphi_G) < (w(\varphi_G) - \log_2 n)/C$ for every $C \leq 10$ falsifies the conjecture.

Rationale (proposer’s reasoning):

Resolution width on Tseitin formulas is governed by expansion of the underlying graph (Ben-Sasson-Wigderson), and persistent H1 lifespans of the clause point cloud quantitatively detect 1-cycles in the parity constraint hypergraph — exactly the topological feature that forces wide clauses. Persistent homology has been applied to neural net loss landscapes and SAT phase transitions but, to our knowledge, never to Tseitin resolution width as a relational, scale-parametrized invariant (likely <5 papers). Because the construction is metric-and-combinatorial (Rips filtration on ± 1 vectors), it does not algebrize, does not relativize (graph-dependent), and is not a poly-time large+useful property of truth tables, dodging the three textbook barriers and the natural-proofs trap.

Taxonomy category: persistent_homology_resolution_width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cde4f07c2972f9c3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 180 instances (6 vertex sizes $\times$ 30 seeds) of odd-charge 3-regular Tseitin formulas, fit $w(\varphi_G)$ vs $L1(\varphi_G) + \log_2 n$ by least squares and compute $C = \max_i w_i / (L1_i + \log_2 n_i)$. *SUPPORTED* iff $C \leq 10$ on every instance AND fit slope ≤ 10 with $R^2 \geq 0.5$; *FALSIFIED* iff any instance has $w_i / (L1_i + \log_2 n_i) > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - persistent homology Tseitin formula resolution width - Vietoris-Rips filtration proof complexity CNF clause incidence - topological data analysis resolution lower bounds expander Tseitin

Top relevant hits considered: - [http://arxiv.org/abs/math/0607634v2] A statistical approach to persistent homology - [http://arxiv.org/abs/2512.07065v1] Persistent Homology-Guided Frequency Filtering for Image Compression - [http://arxiv.org/abs/1904.05081v2] Relative-perfectness of discrete gradient vector fields and multi-parameter persistent homology - [http://arxiv.org/abs/2301.07191v3] A New Construction of the Vietoris-Rips Complex - [http://arxiv.org/abs/2605.00705v1] Cohomological properties of the Vietoris-Rips Complex of a Hypercube Graph - [http://arxiv.org/abs/1203.6786v2] Linear-Size Approximations to the Vietoris-Rips Filtration - [http://arxiv.org/abs/1110.6876v4] Lower bounds for topological complexity - [http://arxiv.org/abs/2005.07866v1] Byzantine-Resilient SGD in High Dimensions on Heterogeneous Data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        if pivot == 0:
```

```

        raise ValueError("Singular matrix")
    for j in range(i, n):
        A[i][j] /= pivot
    for k in range(n):
        if k != i and A[k][i] != 0:
            factor = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]

def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def random_3regular_graph(v):
    while True:
        edges = set()
        vertices = list(range(v))
        random.shuffle(vertices)
        for i in range(v):
            for j in range(i+1, v):
                if len(edges) == 2 * v - 3:
                    break
                if (vertices[i], vertices[j]) not in edges and (vertices[j], vertices[i]) not in edges:
                    edges.add((vertices[i], vertices[j]))
        if len(edges) == 2 * v - 3:
            return [sorted(e) for e in edges]

def tseitin_formula(v):
    clauses = []
    variables = list(range(1, v+1))
    for i in range(v):
        clauses.append([variables[i]])
        clauses.append([-variables[i]])
        for j in range(i+1, v):
            clauses.append([variables[i], variables[j]])
            clauses.append([variables[i], -variables[j]])
            clauses.append([-variables[i], variables[j]])
            clauses.append([-variables[i], -variables[j]])
    return clauses

def persistent_homology(clauses, scale):
    n = len(clauses)
    points = [[-1 if i % 2 == 0 else 1 for _ in range(n)] for _ in range(2)]
    H1_lifespans = []
    for t in range(1, int(scale * math.sqrt(2)) + 1):
        distance_matrix = [[math.inf] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                dist = sum((points[0][i] - points[0][j])**2 for k in range(n))

```

```

        if dist <= t**2:
            distance_matrix[i][j] = distance_matrix[j][i] = dist
# Compute the boundary matrix and perform Gaussian elimination
        B = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if distance_matrix[i][j] <= t**2:
                    B[j][i] = 1

        try:
            gaussian_elimination(B)
        except ValueError:
            continue
# Count the number of non-zero rows to get H1 dimension
        H1_dimension = sum(1 for row in B if any(row))
        H1_lifespans.append(H1_dimension)
    return sum(H1_lifespans)

def resolution_width(clauses, max_width):
    def dpll(clause_set, assignment, model):
        if not clause_set:
            return True
        unit_clause = next((c for c in clause_set if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if dpll(clause_set - {c for c in clause_set if literal in c}, new_assignment, model):
                return True
            new_assignment[literal] = False
            if dpll(clause_set - {c for c in clause_set if -literal in c}, new_assignment, model):
                return True
            return False
        literals = [l for l in range(1, max_width+1) if l not in assignment]
        literal = random.choice(literals)
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll(clause_set - {c for c in clause_set if literal in c}, new_assignment, model):
            return True
        new_assignment[literal] = False
        if dpll(clause_set - {c for c in clause_set if -literal in c}, new_assignment, model):
            return True
        return False

    max_width_found = 0
    for width in range(1, max_width+1):
        if dpll(set(tuple(c) for c in clauses), {}, {}):
            max_width_found = width
        else:
            break
    return max_width_found

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 0
    metric_values = []
    instances_tested = 0

```

```

conjecture_holds = True
counterexample = ""

for v in [6, 8, 10, 12, 14, 16]:
    if v * 3 // 2 > 30:
        break
    n_max = max(n_max, v * 3 // 2)
    clauses = tseitin_formula(v)
    L1 = persistent_homology(clauses, 2 * math.sqrt(3))
    w = resolution_width(clauses, 6)
    metric_values.append(w / (L1 + math.log(v * 3 // 2)))
    instances_tested += 1

if len(metric_values) < 30:
    return {
        "metric_name": "w(L1+log n)",
        "metric_value": sum(metric_values) / len(metric_values),
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean = sum(metric_values) / len(metric_values)
std_dev = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
C_star = max(metric_values)

if C_star > 10:
    conjecture_holds = False
    counterexample = f"C*={C_star} > 10"

return {
    "metric_name": "w(L1+log n)",
    "metric_value": mean,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and max(r["metric_value"] for r in results) > 10:
        print(f"RESULT: NOT SUPPORTED counterexample={counterexample}")

```

```

    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"C* exceeds 10\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05f05e5b.py", line 185, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05f05e5b.py", line 148, in run_trial
    L1 = persistent_homology(clauses, 2 * math.sqrt(3))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05f05e5b.py", line 77, in persistent_homology
    points = [[-1 if i % 2 == 0 else 1 for _ in range(n)] for _ in range(2)]
               ^
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an UnboundLocalError in persistent_homology before any instances were evaluated, so neither the support nor falsification condition can be assessed. | next: Fix the scoping bug in persistent_homology (the list comprehension references an undefined loop variable 'i') and rerun the 180-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	78568
2	propose	claude_max	opus	0	0	154960
3	propose	claude_max	opus	0	0	21700
4	preregistration	claude_max	opus	0	0	11472
5	novelty	claude_max	opus	0	0	4971
6	novelty	claude_max	opus	0	0	8702
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19078
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18694

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18355
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21688
11	judge	claude_max	opus	0	0	4688

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 362876 ms total latency. Provider mix: {'claude_max': 7, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/c6142ec20dd6.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c6142ec20dd6.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c6142ec20dd6.tar.gz (if generated)